Sistemi operativi

Raccolta Completa: 3 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

1. Che cos’è il Sistema Operativo (SO)?

Un computer, senza software, è semplicemente un insieme inerte di componenti elettronici e meccanici (chip di silicio, cavi, circuiti, ventole).

Il **Sistema Operativo (SO)** è il **software di base** fondamentale che fa da **intermediario** (o strato *“cuscinetto”*) tra:

1. L’**utente** e i suoi programmi (**software applicativo**).
2. La parte fisica della macchina (**hardware**).

```mermaid
flowchart TD
    subgraph UTENTI["👤 UTENTI"]
        U["Utente finale / Operatore"]
    end

    subgraph SW_APP["📦 SOFTWARE APPLICATIVO"]
        APP["Browser, Word, Videogiochi, App personalizzate..."]
    end

    subgraph SW_BASE["⚙️ SOFTWARE DI BASE"]
        COMP["Compilatori & Librerie di sistema"]
        OS["SISTEMA OPERATIVO (Kernel, Gestione risorse)"]
    end

    subgraph HW["💻 HARDWARE"]
        H["CPU • Memorie (RAM/ROM)<br/>Dischi/SSD • Bus • GPU • Periferiche I/O"]
    end

    UTENTI <--> SW_APP
    SW_APP <--> SW_BASE
    SW_BASE <--> HW

    style UTENTI fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
    style SW_APP fill:#fff8e1,stroke:#fbc02d,stroke-width:2px;
    style SW_BASE fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
    style HW fill:#ffe0b2,stroke:#f57c00,stroke-width:2px;
```

Distinzione fondamentale del Software

Tipologia	Definizione	Esempi
Software di base	Insieme di programmi **indispensabili** per far funzionare l'hardware e permettere l'esecuzione di qualsiasi altro programma.	Sistemi operativi (Windows, Linux, macOS, Android), compilatori, interpreti, driver e librerie di sistema.
Software applicativo	Programmi creati per risolvere specifici problemi dell'utente o svolgere compiti particolari.	Fogli di calcolo (Excel), browser web (Chrome), videogiochi, elaboratori di testo (Word).

> **Regola d'oro**
>
> L'assenza del sistema operativo rende **totalmente inutilizzabile** qualsiasi computer, server, smartphone o tablet!

---

2. Il Principio di Astrazione e la Macchina Astratta

Grazie al sistema operativo, l'utente e i programmatori **non hanno bisogno di conoscere l'architettura interna** del dispositivo.

Ad esempio:

- Non devi sapere se i dati sono scritti su un hard disk magnetico SATA, su un velocissimo SSD NVMe o su una pennetta USB.
- Non devi sapere se la stampante collegata è laser o a getto d'inchiostro, o come dialogano i suoi transistor.
- Non devi conoscere le specifiche elettriche della scheda di rete o quanti nanometri misurano i transistor della CPU.

Il sistema operativo nasconde tutti questi dettagli tecnici complessi e presenta una **macchina astratta** (o virtuale):

- **Macchina Fisica**: complessa, varia per ogni computer, richiede di gestire segnali elettrici, registri hardware, temporizzazioni e indirizzi esadecimali.
- **Macchina Astratta**: un modello uniforme, pulito e intuitivo, dotato di tutte le risorse necessarie pronte all'uso.

🚗 Le Due Metafore Efficaci

> **La Metafora dell'Automobile**
>
> Per guidare un'auto non hai bisogno di sapere come funzionano i pistoni, le valvole a camme o l'impianto di iniezione elettronica. Ti basta conoscere l'**interfaccia comandi**: volante, pedali e cambio. Il sistema operativo è la "plancia di comando" del computer: offre comandi standardizzati e gestisce lui tutta la "meccanica" interna.

> **La Metafora del Maggiordomo**
>
> Il sistema operativo si comporta come un **bravo maggiordomo**, sempre pronto a rispondere alle nostre richieste: noi diciamo cosa vogliamo fare (es. *"apri questa pagina"* o *"stampa questo documento"*) e lui attiva silenziosamente le risorse hardware adeguate per compiere l'azione, senza costringerci a scendere in cantina a muovere le valvole!

---

3. I Due Obiettivi Primari del Sistema Operativo

Un sistema operativo deve perseguire due scopi essenziali:

1. **Facilità d'uso (Efficacia / Trasparenza)**:
   - Permettere a qualunque utilizzatore (anche non esperto) di sfruttare le risorse hardware e software in modo immediato, trasparente e naturale.
2. **Ottimizzazione delle risorse (Efficienza / Politiche di Gestione)**:
   - Gestire e ripartire le risorse fisiche (che sono **limitate e contese**) tra le diverse applicazioni in esecuzione, evitando sprechi, rallentamenti o blocchi di sistema.

---

4. La Gestione delle Attività: Processi e Multitasking

Che cos'è un Processo?

Spesso usiamo le parole *programma* e *processo* come sinonimi, ma in informatica c'è una netta differenza:

```
flowchart TD
    PROG["💾 PROGRAMMA (Entità Passiva)<br/>Salvato su disco (SSD/Hard Disk)"]
    PROG -- "Caricato in RAM ed eseguito dalla CPU" --> PROC["⚡ PROCESSO (Entità A

    style PROG fill:#eceff1,stroke:#607d8b,stroke-width:2px;
    style PROC fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
```

Caratteristica	Programma	Processo
Natura	Entità **passiva**	Entità **attiva**
Dove si trova?	Memoria secondaria (SSD / Hard Disk)	Memoria centrale (RAM)
Cosa fa?	Codice e istruzioni salvate su file	Istruzioni in esecuzione da parte della CPU

> **Definizione Formale**
>
> **Ogni applicazione (programma) in esecuzione su una macchina prende il nome di PROCESSO.**

Che cos'è il Multitasking?

I moderni sistemi operativi sono **multitasking**: consentono l'esecuzione contemporanea (o con alternanza così rapida da sembrare simultanea) di più programmi.

In un sistema multitasking, molte applicazioni competono contemporaneamente per le stesse risorse hardware. Le risorse più contese sono:

- **CPU**: per eseguire il calcolo delle istruzioni.
- **RAM**: per conservare codice e dati attivi.
- **Memoria di massa (SSD / HDD)**: per caricare o salvare file.
- **Scheda di rete (Wi-Fi / Ethernet)**: per scambiare pacchetti su Internet.
- **GPU (Graphics Processing Unit)**: circuito elettronico dedicato alla grafica digitale, al rendering 3D, all'elaborazione video e alle animazioni.

---

5. Due Situazioni Critiche Gestite dal SO

Il sistema operativo interviene continuamente con le sue **politiche di gestione** razionale per risolvere due problemi critici:

```
flowchart TD
    subgraph S1["1. Programmi Grandi (Memoria Insufficiente)"]
        A["App molto pesante<br/>(es. Video Editor, IDE)"] --> B["Supera lo spazio d
        B --> C["SWAPPING<br/>(Carica/Scarica parti tra RAM e SSD)"]
    end

    subgraph S2["2. Conflitti di Risorse (Attività Simultanee)"]
        D["Download di un file<br/>+ Stampa documento<br/>+ Riproduzione video"] -->
        E --> F["ARBITRAGGIO DEL SO<br/>(Evita collisioni e blocchi)"]
    end
```

1. **Esecuzione di programmi pesanti (Tecnica di Swapping)**:
    - Se un software applicativo ha dimensioni superiori alla RAM disponibile (ad esempio un software di grafica o modellazione 3D), il SO non va in blocco: carica in RAM solo le porzioni di programma che servono in quel momento e scarica su disco quelle inutilizzate. Questo continuo scambio tra RAM e memoria secondaria si chiama **swapping**.
2. **Concorrenza di più attività (Prevenzione dei conflitti)**:
    - Se stiamo ascoltando musica, scaricando un file e stampando contemporaneamente, tutti questi processi chiedono l'uso della CPU e della memoria. Il SO coordina i turni e le

priorità per far sì che nessuno si blocchi e non ci siano conflitti di accesso alle medesime risorse.

---

6. Monitoraggio Reale: La Gestione Attività (Task Manager)

Nei sistemi Windows, lo strumento fondamentale per osservare l'operato del sistema operativo è il pannello **Gestione attività** (*Task Manager*, apribile con `Ctrl + Shift + Esc`):

- **Scheda Processi**: mostra la lista di tutte le applicazioni e dei processi in background, indicando per ciascuno la percentuale di CPU, RAM, Disco, Rete e GPU consumata in tempo reale.
- **Scheda Prestazioni**: mostra grafici dinamici sullo stato di carico complessivo dell'hardware (frequenza CPU, memoria occupata/disponibile, velocità di lettura/scrittura su disco, traffico Wi-Fi e carico GPU).

1. Visione d’Insieme: Nucleo e Guscio

Un sistema operativo può essere idealmente suddiviso in due grandi sezioni complementari:

```
flowchart TD
    U["👤 UTENTE"] <--> SH["🐚 SHELL (Interfaccia esterna / Guscio)<br/>Interpreta
    SH <--> K["⚙️ KERNEL (Nucleo interno protetto)<br/>Sempre residente in RAM • Ge
    K <--> HW["💻 HARDWARE (CPU, RAM, Dischi, Schede)"]

    style U fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
    style SH fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
    style K fill:#fff3e0,stroke:#f57c00,stroke-width:2px;
    style HW fill:#ffebee,stroke:#d32f2f,stroke-width:2px;
```

---

2. Il Nucleo del Sistema Operativo: Il Kernel

Il **Kernel** (in tedesco *“nocciolo”* o *“seme”*) è la parte centrale e più importante del sistema operativo. È il ponte diretto tra i componenti hardware e i programmi in esecuzione.

Perché il Kernel risiede sempre in RAM?

Un sistema operativo moderno è un software gigantesco (molti gigabyte di file su disco). Se venisse caricato tutto interamente nella memoria RAM:

- Occuperebbe quasi tutta la memoria disponibile;
- Non lascerebbe spazio sufficiente ai programmi applicativi dell’utente.

Per questa ragione:

1. I moduli secondari del SO risiedono su memoria secondaria (SSD/disco) e vengono caricati in RAM **solo quando servono**.
2. **Solo il Kernel è sempre residente in memoria centrale (RAM)**: viene caricato durante la fase di accensione (**bootstrap**) e **non viene mai scaricato**, né spostato su disco (divieto assoluto di *swapping*), perché senza di esso l’elaboratore cesserebbe istantaneamente di funzionare.

> **Il Bootstrap**
>
> Il **bootstrap** è la sequenza di avvio del computer: all'accensione della macchina, il firmware (BIOS/UEFI) individua l'unità di memoria secondaria di avvio e carica il kernel del sistema operativo nella memoria centrale (RAM), avviando il sistema.

I 6 Compiti del Kernel per ogni Processo

Quando lanciamo un'applicazione, il kernel crea un **processo** ed esegue sei operazioni fondamentali:

```
flowchart TD
    P1["1. Assegna il tempo di CPU"] --> P2["2. Assegna spazio in RAM e risorse HW/S
    P2 --> P3["3. Stabilisce la priorità (multitasking)"]
    P3 --> P4["4. Carica il codice macchina eseguibile"]
    P4 --> P5["5. Avvia l'esecuzione del processo"]
    P5 --> P6["6. Gestisce ed esaudisce le richieste (System Call)"]
```

1. **Assegna il tempo di CPU**: decide quanto tempo di calcolo concedere al processo.
2. **Assegna lo spazio di memoria**: riserva l'area di RAM necessaria per dati e istruzioni.
3. **Stabilisce la priorità**: determina l'importanza del processo rispetto agli altri processi attivi nel sistema.
4. **Carica il codice macchina**: trasferisce il file eseguibile da disco/SSD alla memoria centrale.
5. **Avvia l'esecuzione**: passa il controllo del Program Counter della CPU alla prima istruzione.
6. **Interagisce con le richieste**: risponde a tutte le chiamate di servizio avanzate dal processo mentre è in esecuzione (es. richiesta di leggere un file o connettersi a Internet).

Gestione della Memoria e Virtualizzazione

Il kernel è responsabile di garantire che **nessun processo interferisca con la memoria usata da un altro processo** (protezione e isolamento reciproco).

- Applica tecniche di **virtualizzazione della memoria**, facendo credere a ciascuna applicazione di avere a disposizione un banco di memoria continuo e riservato.

- Decide in ogni istante quanta memoria concedere a ciascun processo e può assegnare porzioni di memoria condivisa quando più processi devono scambiarsi informazioni.

Lo Scheduler (Schedulatore)

In un sistema multitasking, la CPU deve dividere la sua attenzione tra decine o centinaia di processi attivi. Il componente del kernel incaricato di questo compito è lo **Scheduler**:

- **Scandisce i tempi di esecuzione** assegnando la CPU a rotazione (*time slice*).
- **Gestisce lo stato dei processi**:
  - `Ready` (Pronto): il processo è pronto in coda e attende che la CPU si liberi.
  - `Running` (In esecuzione): il processo sta usando la CPU in questo preciso istante.
  - `Waiting / Blocked` (In attesa): il processo è fermo in attesa di un evento esterno (es. dati da disco, pressione di un tasto, risposta dalla rete).

```
↔stateDiagram-v2
    direction TB
    [*] --> Pronto: Nuovo processo
    Pronto --> InEsecuzione: Assegna CPU
    InEsecuzione --> Pronto: Tempo scaduto
    InEsecuzione --> InAttesa: Attesa I/O
    InAttesa --> Pronto: I/O completato
    InEsecuzione --> [*]: Concluso
```

> **Perché lo Scheduler è geniale?**
>
> La lettura dei dati da un SSD o da un hard disk è infinitamente più lenta della velocità della CPU. Se la CPU dovesse “aspettare senza far nulla” ogni volta che un programma carica un file, il computer sembrerebbe congelato. Lo **scheduler** sospende il processo in attesa e assegna immediatamente la CPU a un altro processo pronto: in questo modo la CPU lavora sempre al massimo dell’efficienza!

Riepilogo dei Compiti Complessivi del Kernel

1. **Gestione della CPU**: scheduling e alternanza dei processi pronti per l'elaborazione.
2. **Gestione delle interruzioni (Interrupt)**: gestione tempestiva dei segnali di allarme o di richiesta prioritari inviati dall'hardware o dal software.
3. **Gestione della memoria**: allocazione dello spazio di RAM per ogni programma.
4. **Controllo dell'hardware tramite i Driver**: utilizzo di moduli software specifici (**driver**) per dialogare correttamente con ogni singola periferica.
5. **Controllo del ciclo di vita dei processi**: monitoraggio dello stato di avanzamento (*ready, running, waiting, terminated*).
6. **Sincronizzazione e comunicazione (IPC)**: meccanismi di scambio dati ordinato e sicuro tra processi cooperanti.

---

3. Il Guscio del Sistema Operativo: La Shell

Se il kernel è il nucleo interno protetto, la **Shell** (in inglese *"guscio"* o *"conchiglia"*) è l'**involucro esterno**: è l'**interfaccia** mediante la quale l'utente può dialogare e operare sul sistema.

La shell opera come un **interprete dei comandi**:

1. Rimane in attesa di un'istruzione digitata o cliccata dall'utente.
2. Esegue il **parsing** del comando.
3. Se il comando è corretto, ordina al kernel di eseguirlo.

> **Techwords: Che cos'è il Parsing?**
>
> Il **parsing** (o analisi sintattica) è il processo informatico che esamina una sequenza di caratteri in ingresso per verificare se rispetta le regole grammaticali e sintattiche del linguaggio/shell prima di eseguirla.

---

4. Le Tre Tipologie di Shell a Confronto

Nel corso dell'evoluzione dell'informatica sono state sviluppate tre tipologie di shell:

```
flowchart TD
    SH["🐚 LE SHELL DEL SISTEMA OPERATIVO"]
    SH --> M["1. Shell a Menu<br/>Scelte guidate da tastiera/mouse (es. BIOS, UEFI)"
    M --> CLI["2. Shell a Linea di Comando (CLI)<br/>Comandi testuali e script (es.
    CLI --> GUI["3. Shell Grafica (GUI)<br/>Desktop, icone e finestre (es. Windows,
```

Tabella Comparativa delle Shell

Caratteristica	Shell a Menu	Shell a Linea di Comando (CLI)	Shell a Interfaccia Grafica (GUI)
Interfaccia	Schermate testuali con opzioni a elenco	Riga di testo (cursore lampeggiante)	Visiva, basata su finestre, icone e puntatore
Modalità di input	Tasti freccia/Invio (e mouse in UEFI)	Tastiera (digitazione rigorosa di comandi)	Mouse, touchpad, tastiera o touchscreen
Facilità per principianti	Media	Bassa (occorre memorizzare i comandi)	Massima (*user-friendly*)
Potenza e Automazione	Limitata alla configurazione	Altissima (scripting e automazione potente)	Media (orientata all'interazione manuale)
Esempi Tipici	**BIOS**, moderni firmware **UEFI**	**Prompt dei comandi** (`cmd`), **PowerShell**, **Bash**	**Windows**, **macOS**, **Ubuntu Desktop**, **Android**, **iOS**

Dettaglio sulle singole tipologie:

1. Shell a Menu

- Presentano elenchi e sottomenu guidati.

- Storicamente presenti nel **BIOS** (gestito solo da tastiera per impostare l'hardware di base della scheda madre).
- Nei moderni computer sono sostituite da **UEFI**, dotato di un'interfaccia a menu più ricca che supporta anche l'uso del mouse e connessioni di rete.

2. Shell a Linea di Comando (CLI - Command Line Interface)

- L'utente immette comandi testuali rispettando una grammatica precisa (comando, opzioni/flag, argomenti).
- Principali ambienti:
  - **Prompt dei comandi (`cmd.exe`)**: la tradizionale shell DOS/Windows.
  - **Microsoft PowerShell**: shell di nuova generazione dotata di un vero e proprio ambiente di sviluppo integrato (**ISE** - *Integrated Scripting Environment*) orientato agli oggetti.
  - **Bash (Bourne-Again Shell)**: la shell testuale standard del mondo Linux e Unix.
- *Perché si usa ancora?* Perché consente agli amministratori di sistema di creare **script di automazione** capaci di svolgere compiti complessi su centinaia di computer in pochi istanti.

3. Shell a Interfaccia Grafica (GUI - Graphical User Interface)

- È l'interfaccia moderna più diffusa, pensata per essere **user friendly** (amichevole per l'utente comune).
- Si basa sulla **metafora del Desktop** (la "scrivania"): lo schermo simula un piano di lavoro dove si dispongono fogli, cartellette e strumenti.
- **Elementi costitutivi (paradigma WIMP)**:
  - **Finestre** (*Windows*): riquadri che contengono le app in esecuzione o le cartelle;
  - **Icone** (*Icons*): piccole immagini simboliche che rappresentano file, app o cestino;
  - **Menu e Barre degli strumenti**: elenchi di pulsanti per attivare opzioni rapide;
  - **Puntatore** (*Pointer*): freccia mossa dal mouse per puntare e cliccare;
  - **Finestre di dialogo**: riquadri popup interattivi per percorsi guidati, conferme o messaggi di errore.
- **Cenni Storici**: Il primo sistema operativo commerciale di massa con interfaccia grafica e mouse fu introdotto da **Apple con il Macintosh nel 1984**, rivoluzionando l'industria dell'informatica.

---

1. Il Modello a Ring (Architettura a “Buccia di Cipolla”)

Un sistema operativo moderno è un programma immenso e sofisticato. Per evitare che sia caotico ed esposto a errori critici, la sua struttura software è divisa in **moduli** organizzati in una serie di anelli concentrici nidificati, detti **Ring**.

Questo schema prende il nome di **modello a ring** o **modello a buccia di cipolla**:

```
flowchart TD
    subgraph L_USER["👤 LIVELLO UTENTE (User Space)"]
        U["Applicazioni Utente (Word, Browser, Giochi)"]
    end

    subgraph L_SHELL["🐚 LIVELLO INTERFACCIA"]
        SH["Shell (GUI, CLI, Menu)"]
    end

    subgraph L_OS["⚙️ MODULI DEL SISTEMA OPERATIVO"]
        R4["Ring 4: File Management System (File System)"]
        R3["Ring 3: Device Management System (Periferiche I/O)"]
        R2["Ring 2: Memory Management System (RAM e Paginazione)"]
        R1["Ring 1: Kernel (Nucleo a contatto con l'hardware)"]
    end

    subgraph L_HW["💻 HARDWARE"]
        HW["CPU • Registri • Banchi RAM • Controller • Bus"]
    end

    U --> SH
    SH --> R4
    R4 --> R3
    R3 --> R2
    R2 --> R1
    R1 --> HW

    style L_USER fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
    style L_SHELL fill:#ede7f6,stroke:#7e57c2,stroke-width:2px;
    style R4 fill:#e8f5e9,stroke:#43a047,stroke-width:2px;
    style R3 fill:#fff9c4,stroke:#fbc02d,stroke-width:2px;
    style R2 fill:#ffe0b2,stroke:#fb8c00,stroke-width:2px;
    style R1 fill:#ffcdd2,stroke:#e53935,stroke-width:2px;
    style L_HW fill:#cfd8dc,stroke:#455a64,stroke-width:2px;
```

---

2. Il Principio della Macchina Astratta a Livelli

In questo modello:

- **Ogni anello rappresenta una macchina astratta differente**, costituita dai moduli software di quell'anello più tutti gli anelli a esso interni, fino all'hardware.
- **Regola di comunicazione gerarchica**: I moduli di un anello esterno **non dialogano mai direttamente con l'hardware**, bensì possono richiamare **esclusivamente i moduli dell'anello immediatamente più interno**. Solo l'anello più interno (il Kernel) tocca fisicamente i circuiti della macchina!

> **Regola di Comunicazione Gerarchica**
>
> **Anello Esterno** ➔ comunica *solo con* ➔ **Anello Interno** ➔ ... ➔ **Hardware** *(È rigorosamente vietato saltare livelli intermedi o accedere direttamente all'Hardware)*

> **Il Grande Vantaggio della Modularità**
>
> Poiché i sistemi operativi devono funzionare su configurazioni hardware diversissime (processori Intel, AMD, ARM, schede grafiche e dischi di decine di marche diverse), l'organizzazione modulare permette di **sostituire o aggiornare un solo componente** (ad esempio installando un nuovo **driver**) senza dover riscrivere tutto il sistema operativo!

---

3. I 4 Ring del Sistema Operativo in Dettaglio

Analizziamo i compiti specifici di ciascuno dei 4 anelli del sistema operativo (dal più interno al più esterno):

Ring	Modulo del Sistema Operativo	Funzioni Principali e Responsabilità
Ring 1	**Kernel (Nucleo)**	È lo strato più vicino alla macchina fisica. Gestisce la CPU, scandisce l'avanzamento dei processi (scheduling), gestisce gli **interrupt** hardware/software e controlla il funzionamento base dei driver.
Ring 2	**Memory Management System** *(Gestione Memoria)*	Gestisce la memoria RAM: tiene traccia in ogni istante di quali aree sono libere e quali occupate; **traduce l'indirizzo logico** del programma in **indirizzo fisico** reale; alloca e rilascia la memoria al termine dei processi.
Ring 3	**Device Management System** *(Gestione Periferiche)*	Gestisce la comunicazione con le periferiche (tastiera, mouse, monitor, stampante, schede); controlla il flusso di dati e segnali di I/O; assegna l'accesso ai dispositivi ed evita che due programmi vadano in conflitto.
Ring 4	**File Management System** *(comunemente **File System**)*	Gestisce l'organizzazione logica dei dati salvati sulla memoria secondaria (SSD, HDD, USB) sotto forma di **file e cartelle**; gestisce creazione, lettura, scrittura, cancellazione e backup dei file, astraendo la posizione fisica dei settori su disco.

Oltre questi 4 anelli risiedono la **Shell** (l'interprete comandi / interfaccia grafica) e infine l'**Utente** con le sue applicazioni.

---

4. User Mode vs Kernel Mode

Per motivi di **sicurezza**, **stabilità** e **affidabilità**, la CPU del computer e il sistema operativo lavorano alternandosi tra due distinte modalità di esecuzione:

```
flowchart TD
    subgraph UM["👤 USER MODE (Modalità Utente)"]
        direction TB
        App["Programmi applicativi (Word, Chrome, Giochi)"]
        PrivU["Privilegi bassi / limitati • Memoria isolata e protetta"]
    end

    subgraph KM["⚙️ KERNEL MODE (Modalità Supervisore)"]
        direction TB
        Kern["Kernel e Driver di sistema"]
        PrivK["Privilegi massimi • Accesso completo a HW, RAM e CPU"]
    end

    App -- "1. Invoca SYSTEM CALL (Richiesta servizio)" --> Kern
    Kern -- "2. Esegue operazione e restituisce il controllo" --> App

    style UM fill:#e3f2fd,stroke:#1976d2,stroke-width:2px;
    style KM fill:#ffebee,stroke:#c62828,stroke-width:2px;
```

Confronto Diretto tra le Due Modalità

Proprietà	User Mode (Modalità Utente)	Kernel Mode (Modalità Supervisore)
Chi la usa?	I programmi applicativi dell'utente (Word, Browser, player video).	Il Kernel del sistema operativo e i driver essenziali.
Livello di Privilegi	**Basso / Limitato**.	**Massimo / Totale**.
Accesso all'Hardware	**Vietato**: non può accedere direttamente a porte I/O, dischi o schede.	**Completo e Diretto** a tutte le periferiche e componenti.
Accesso alla Memoria	Può leggere/scrivere **solo nella propria area di memoria privata**.	Può accedere a **tutta la memoria fisica** del sistema.
Istruzioni CPU	Solo istruzioni standard non pericolose.	Tutte le istruzioni, comprese quelle privilegiate di gestione CPU.
Cosa succede in caso di Crash?	Si chiude solo l'applicazione con un messaggio di errore (il resto del PC continua a funzionare).	**Crash totale del sistema** (la classica "Schermata Blu" o Kernel Panic).

Perché non eseguire tutto in Kernel Mode?

Se un semplice browser web o un programma scritto male girasse in Kernel Mode, un banale bug (o un virus) potrebbe sovrascrivere la memoria di altri programmi o distruggere file del sistema operativo, facendo **crollare l'intero computer**! Mantenere le applicazioni confinate in User Mode garantisce che un crash di un programma non comprometta la sicurezza e la stabilità degli altri processi.

Comportamento della CPU in Kernel Mode

Quando la CPU opera in **kernel mode**, esegue operazioni critiche che **non possono essere interrotte o terminate** a piacere dai normali processi, assicurando che le risorse condivise

non rimangano in uno stato corrotto o inconsistente.

---

5. Il Ponte tra i Due Mondi: Le System Call

Se un'applicazione in *User Mode* non può toccare l'hardware, **come fa a salvare un file sul disco o a inviare un messaggio tramite Wi-Fi?**

Lo fa per mezzo delle **System Call** (chiamate al sistema operativo)!

> **Definizione di System Call**
>
> Una **System Call** è un'istruzione speciale con cui un programma applicativo (in *User Mode*) chiede al sistema operativo di eseguire un'operazione privilegiata per suo conto.

Sequenza tipica di una System Call:

```
flowchart TD
    S1["📦 1. App Utente (User Mode)<br/>Invia una System Call (es. 'Salva file')"]
    S2["⚡ 2. Trap della CPU<br/>Commutazione: User Mode ➔ Kernel Mode"]
    S3["⚙️ 3. Kernel del SO (Kernel Mode)<br/>Verifica autorizzazioni e sicurezza"]
    S4["💾 4. Hardware (SSD / Disco)<br/>Scrittura fisica dei dati sul supporto"]
    S5["⚙️ 5. Kernel del SO<br/>Prepara i dati e l'esito dell'operazione"]
    S6["⚡ 6. Ritorno CPU<br/>Commutazione: Kernel Mode ➔ User Mode"]
    S7["📦 7. App Utente (User Mode)<br/>Riceve conferma di operazione completata"]

    S1 --> S2 --> S3 --> S4 --> S5 --> S6 --> S7

    style S1 fill:#e3f2fd,stroke:#1976d2,stroke-width:2px;
    style S2 fill:#fff9c4,stroke:#fbc02d,stroke-width:2px;
    style S3 fill:#ffebee,stroke:#c62828,stroke-width:2px;
    style S4 fill:#ede7f6,stroke:#7e57c2,stroke-width:2px;
    style S5 fill:#ffebee,stroke:#c62828,stroke-width:2px;
    style S6 fill:#fff9c4,stroke:#fbc02d,stroke-width:2px;
    style S7 fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
```

1. **Richiesta**: L’applicazione chiama una funzione di sistema (es. salvare un file).
2. **Commutazione (Trap)**: La CPU rileva la System Call, interrompe l’User Mode e **commuta in Kernel Mode**.
3. **Controllo ed Esecuzione**: Il Kernel verifica se l’operazione è lecita e sicura; se sì, attiva l’hardware (Ring 1) ed effettua l’operazione.
4. **Ripristino**: Il Kernel restituisce il risultato, la CPU **torna in User Mode** e il programma riprende la sua normale esecuzione.
